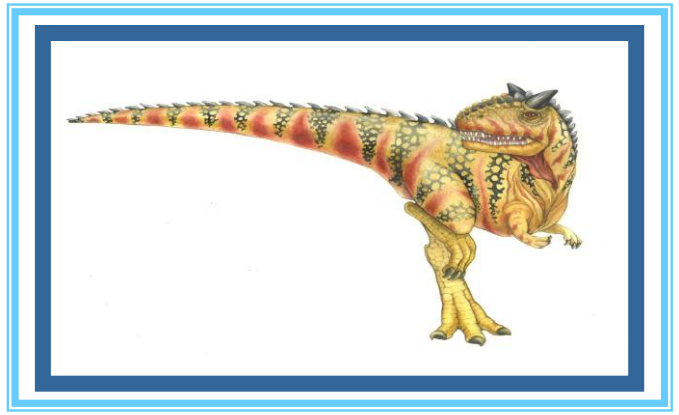
Scheduling – Part 3



Silberschatz, Galvin and Gagne ©2018



- *User-level* and *kernel-level* threads
- It is kernel-level threads-not processes-that are being scheduled by the operating system
- User-level threads are managed by a thread library and the kernel is unaware of them.
- User-level threads must ultimately be mapped to an associated kernel-level thread
- This mapping may be indirect and may use a lightweight process (LWP)
- What are the scheduling issues involving user-level and kernel-level threads?





Thread Scheduling

- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
  - Known as **process-contention scope** (**PCS**) since scheduling competition is within the process
  - Typically done via priority set by programmer
- Does not mean that the thread is actually running on a CPU; this would require the operating system to schedule the kernel thread onto a physical CPU
- Typically, PCS is done according to priority
  - the scheduler selects the runnable thread with the highest priority to run.
  - User-level thread priorities are set by the programmer and are not adjusted by the thread library, although some thread libraries may allow the programmer to change the priority of a thread.





Thread Scheduling

- To decide which kernel thread to schedule onto a CPU, the kernel uses **system-contention scope** (**SCS**)
- Competition for the CPU with SCS scheduling takes place among all threads in the system.
- Systems using the one-to-one model, such as Windows XP, Solaris, and Linux, schedule threads using only SCS.





Pthread Scheduling

- API allows specifying either PCS or SCS during thread creation
  - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
  - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Can be limited by OS – Linux and macOS only allow PTHREAD_SCOPE_SYSTEM





Pthread Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");
      else
         fprintf(stderr, "Illegal scope value.\n");
   }
```





Pthread Scheduling API

```
   /* set the scheduling algorithm to PCS or SCS */
   pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i],&attr,runner,NULL);
   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```

